Geavanceerde computerarchitectuur: lab-2

Lukas Dewachter

February 2024

1 Learning objective

In this lab session we learn what race conditions are and how to prevent this. This principle will be demonstrated by finding the max number in a random array in three different ways.

2 Exercise

2.1 CPU

This is the normal way to do it. You iterate through the array and check if its higher than the max number.

2.2 Atomic

Here we use the 'atomicMax' function to determine the max number through CUDA. This means you will lose time allocating the memory to the GPU. And you can divide the array into blocks by using the GPU's threads in blocks like we did in the last lab.

2.3 Reduction

This method is also done via the GPU so will also use Thread blocks and will lose time to memory allocation. We can calculate the max number by dividing the array into pairs of two to calculate their max then pair this number up with another pair they then compare and so on. By doing this the array gets calculated faster because instead of one critical section (a section which can only be accessed by one thread at the time) being the max number. This method has a critical section per block so there is less waiting period. After each reduction the threads need to synchronise to avoid race conditions.

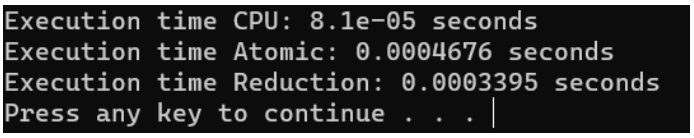


Figure 1: executing speeds for array size 100000

3 Conclusions

When we need to update a certain value whilst executing large operations you need to take in account race conditions. By using the GPU over the CPU we can execute large calculations faster and by using reduction we can lower the executing time because there are more critical sections so less waiting. In the data I have presented the CPU has the fastest time, I think this is because I didn't use a big enough array. I did this because a large array crashed my computer.

